

Report: MoCoDPL - The impact of self-supervision on neuro-symbolic learning

1st Nguyen Anh Thu Tran
CIMeC
University of Trento
Rovereto, Italy
nguyenanhthu.tran@studenti.unitn.it

Abstract—The project aims to leverage the advantages of self-supervision for traditionally supervised neuro-symbolic models. A Momentum Contrast encoder is pretrained and applied to DeepProbLog, a probabilistic logic semantic model. The results indicate that the encoder architecture and hyperparameter choices may play a bigger role than pretraining, especially for concept collapse metrics. The code is available at: <https://github.com/mayleyc/kanddpl-moco>

Index Terms—NeSy, neuro-symbolic, DPL, MoCo, self-supervision, concept collapse, reasoning shortcuts.

I. INTRODUCTION

Neuro-symbolic (NeSy) architectures such as DeepProbLog (DPL) and logic tensor networks (LTN) contain two main components: the perception module, which translates input x into concepts c , and the reasoning module, which deduces label y based on concepts c and the likely rules surrounding the concepts (see Fig. 1). NeSy models are trained using query and world probabilities on examples annotated with labels, though concept annotation is not always available [1]. Reasoning shortcuts may occur when a NeSy model conflates a concept with another due to the lack of clarity in concept formation (e.g. a person crossing the road can be interpreted in the same way as a red light, which may lead to catastrophic results in high-stakes situations).

Meanwhile, self-supervision methods such as Momentum Contrast (MoCo) have been shown to promote meaningful concept representation [3], [7] by contrasting pairs of examples instead of simply generalizing from all of them. This opens up the possibility of applying self-supervision to neuro-symbolic models for higher-quality concepts.

Contributions

This project proposes a pipeline which requires little computational cost (tested on a RTX 3090) based on the Reasoning Shortcuts Benchmark Suite (rsbench) that leverages self-supervision for neuro-symbolic models, and can be applied to similar 2D image classification datasets which contain concept annotations. [1] provides the opportunity to improve on existing implementations through a variety of difficult benchmarks, as well as ready-made training and testing scripts.

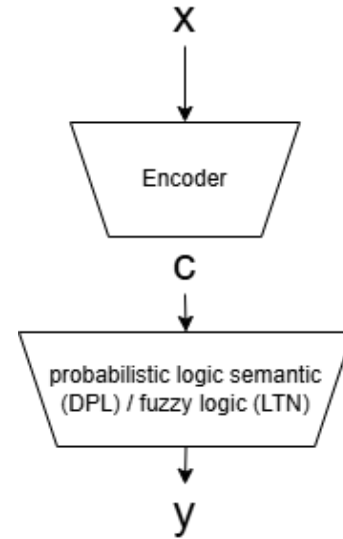


Fig. 1: The basic data flow of neuro-symbolic models such as DPL and LTN

The project, therefore, is greatly influenced by this paper. The code is only slightly modified from the original rsbench.

Research Questions

The project aims to investigate the following question: Does MoCo help improve the performance of DPL on the Kandinsky dataset? Specifically, does it help with preventing concept collapse and allow for better concept representation?

To approach these questions, the original encoder (Encoder-Conv64) was replaced with the MoCo-ViT encoder (vit-conv-small). Two variants of MoCo-DPL were evaluated alongside the DPL model implemented in [1]: *moco-ft* (randomly initialized encoder) and *moco-pre* (pretrained encoder). This serves to investigate whether performance may be improved by MoCo contrastive learning or simply due to a different encoder. The variants will be tested on the "patterns" task of Kand-Logic only, due to time limitations. Three main metrics: label F1, concept F1 and concept collapse (CIs) will be used to compare their performance with the baseline.

II. METHODS

A. Dataset

Kand-Logic (or “Kandinsky” in the code) is a 2D image classification dataset based on the paintings by Wassily Kandinsky, and in turn, the Kandinsky Patterns task by [6]. In Kand-Logic, given an image with three groups of geometric shapes (square, circle, triangle), possibly of different colors (red, blue, yellow), the “patterns” task requires the model to discern whether the third group satisfies the rule followed by the first two groups. Examples of the rules during dataset creation include “all items have the same color” or “exactly two items are of the same shape”. The shapes and colors make up the 6 true concepts annotated for each group, along with the true label y indicating whether the rule is followed by the third group.

The dataset was selected for two reasons: First, it is less computationally complex than other datasets in the benchmark such as CLE4EVR and SDD-OIA, while also satisfying conditions for complex and ambiguous concepts [1]. This provides enough difficulty to showcase improvements in concept representation. Second, image classification tasks are compatible with the original implementation of MoCo.

The dataset contains 3 splits: train (4000), validation (1000) and test (1000). Because the current dataset generator (rssgen) creates vastly different labeling from the original one, no out-of-distribution (OOD) splits were included in the experiments. Preprocessing involves splitting the images into 3 images of size 64x64 (3 groups) without resizing, and flattening them into vectors. For the MoCo-ViT pretraining of 1000 epochs, images were normalized to the default ImageNet values of mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225], for the MoCo model was pretrained on ImageNet.

B. Models

DeepProbLog (DPL) is a state-of-the-art NeSy model (Manhaeve et al., 2018). The current implementation used is from [5]. The default encoder consists of 4 convolutional layers and 2 linear layers at the end. Concepts or features are derived from the encoder, and then query and world probabilities are calculated to inform its predictions. In [1], DPL achieves a higher label F1 score than LTN but underperforms on concept F1 and concept collapse. This may be due to overly reductive representations of shape and color concepts (not capturing all true concepts). On the other hand, MoCo is reported to have improved image representation in real-life applications [3], [7] and has the advantage of moderate batch sizes (compared to other contrastive learning methods such as SimCLR). It employs a momentum encoder to assign examples into contrastive pairs. The latest version of MoCo, v3 [2] is used. The project expects an improvement in DPL’s concept quality using a pretrained encoder with better feature extraction.

MoCo-ViT, the new encoder introduced to DPL in this experiment, is a variant of the MoCo encoder which consists of a convolutional stem (ConvStem) and a MoCo-VisionTransformer (MViT) encoder. It encodes the image by

passing it through ConvStem, then through MViT’s modified 2D position embedding (originally, ViT employs 1D position encoding) which helps the transformer component assign order to the tokens. The project simply modifies MoCo-ViT by adding a linear layer after the encoding process to convert its output from `[batch_size, num_tokens, embed_dim]` to 18 chunks of `[batch_size, 2]` for compatibility with DPL.

C. Metrics

The 3 main metrics reported in [1] are: Label macro-F1, concept macro-F1 and concept collapse (CIs). Macro-F1 is defined as:

$$\text{mF1}(C) = \frac{1}{N} \sum_{i=1}^N \text{F1}(C_i) \quad (1)$$

$$\text{F1}(C_i) = \frac{2 \cdot \text{Precision}(C_i) \cdot \text{Recall}(C_i)}{\text{Precision}(C_i) + \text{Recall}(C_i) + \varepsilon} \quad (2)$$

$$\text{Precision}(C_i) = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i + \varepsilon} \quad (3)$$

$$\text{Recall}(C_i) = \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i + \varepsilon} \quad (3)$$

Where:

- N is the total number of concepts or labels (6 and 2 respectively),
- C_i is the i -th concept or label,
- $\text{F1}(C_i)$ is the binary F1 score for concept/label C_i ,
- TP_i , FP_i , and FN_i refer to the true positives, false positives, and false negatives for C_i ,
- ε is a small value added for numerical stability.

Both label and concept macro-F1 are used as the base metrics, since they provide a more robust assessment of performance than accuracy in multiclass tasks.

The next metric, concept collapse, measures how well the predicted concepts encapsulate the true concept space. It utilizes concept matrix coverage, where coverage is the fraction of predicted concepts that correspond to at least one true concept. For Kand-Logic, [1] computes the collapses for shapes and colors in base 10 (calculated from base 3), reducing the need for further conversions. Separate confusion matrices for shapes and colors are also used to calculate the corresponding CI for each concept. The detailed formula is in [1].

In addition, label and concept accuracy will also be reported for a more comprehensive assessment of performance. This makes up a total of seven metrics. All metrics in ?? are calculated as means and standard deviations (SDs) of the 5 seeds. Qualitative examples of reasoning shortcuts will also be extracted from results of the last seed from each variant (i.e. different models and hyperparameters).

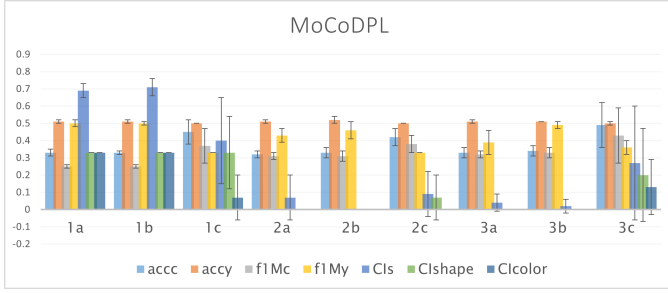


Fig. 2: Items are numbered 1-3 (in order: *nomoco*, *moco-ft* and *moco-pre*) and a-c (a and b are two separate runs with $c_sup=0$; c means $c_sup=0.5$).

D. Experimental Setup

Experiments were conducted with Python 3.8 on a NVIDIA GeForce RTX 3090. 3 model variants were used in the experiments: *nomoco* (original), *moco-ft* (encoder replaced with randomly initialized vit-conv-small encoder) and *moco-pre* (same as *moco-ft*, but with the encoder pretrained on 1000 epochs). All models were then trained on Kand-Logic for 100 epochs.

The pretrained encoder is from a MoCo-ViT model trained on the Kandinsky dataset itself for 1000 epochs instead of 3000, since Kandinsky is a smaller dataset (6000 images) than CIFAR-100 (60000), the largest dataset used for MoCov3 transfer learning. As a rule of thumb, using a smaller number of epochs helps prevent overfitting [8].

The main experiment was performed on 2 runs x 2 concept supervision values x 3 model variants, each on 5 seeds: 123, 456, 789, 1011, 1213, unlike the original benchmark tested on 10 seeds, and only one run with a concept supervision value of 0.5 is performed on each variant due to time limitations. Hyperparameter sweeping was minimal, concerning only the number of epochs (n_epochs) and concept supervision (c_sup). For choosing n_epochs , the project includes an informal experiment, the results of which support the use of 100 epochs over the original 10 (This may have been an oversight; see **Limitations**). Another experiment was carried out to investigate the role of concept supervision on all 3 model variants. Preliminary results on the original model showed that the concept matrices (shape and color) do not differ significantly between c_sup values of 0.5 and 1. Therefore, a value of 0.5 was selected for runs with concept supervision ([1] did not state explicitly which value to use; default was 0). Other hyperparameters were kept the same, such as the learning rate (0.001) with an exponential decay of 0.9 to ensure comparable results. All training runs are tracked using wandb loggers.

III. RESULTS

A. Scores

In general, the MoCo-ViT encoder is highly effective in preventing concept collapse (in most cases, reducing it to near zero), although there is no significant difference between

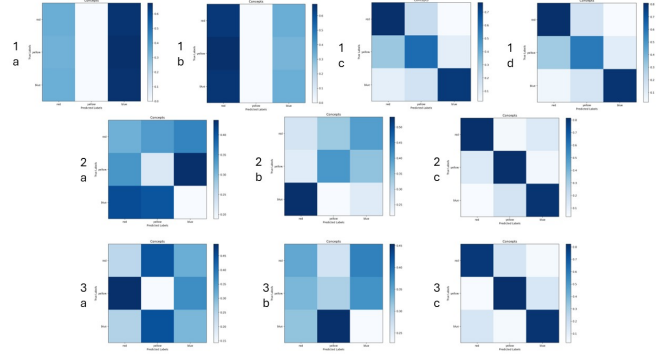


Fig. 3: Color confusion matrices. Graphs are numbered 1-3 (in order: *nomoco*, *moco-ft* and *moco-pre*) and a-d (a and b are two separate runs with $c_sup=0$; c means $c_sup=0.5$; d is $c_sup=1$ for *nomoco* only. Confusion matrices 1d are provided as rationale behind the choice of testing other variants on $c_sup=0.5$).

moco-ft and *moco-pre* in terms of concept accuracy and metrics (Fig. 2). The exception is when c_sup is set at 0.5, leading to a noticeable improvement from $c_sup=0$, and broadens the gap between *moco-ft* and *moco-pre* (up to 0.07).

On the other hand, all models show poor label accuracy and F1. This is also shown during training, where these metrics remain stagnant. It is important to note, however, that the figures close to 50% are most likely due to the model labeling almost every item as false (see Tab. I). This may suggest the lack of rule-deducing ability, considering how the variants all perform differently in terms of concept metrics.

The role of the c_sup hyperparameter is also interesting to note. First, a c_sup of 0.5 did not improve any metrics except for concept accuracy and F1. This may imply that concept supervision does not actually prevent concept collapse by covering the world matrix; instead, there may be biases towards or against certain colors or shapes. For instance, a closer look at the 2c and 3c shape confusion matrices (see Fig. 4) reveals that the model does not generalize circles as well as other shapes, even with a stronger encoder. (In fact, 1c and 1d imply that DPL may inherently have a strong bias towards certain shapes despite concept supervision.) Second, a c_sup of 0.5 leads to more fluctuations in performance, as evidenced by larger SDs across all three model variants.

B. Confusion Matrices

The confusion matrices (3, 4) suggest that shapes might be a harder concept for DPL than colors: with $c_sup > 0.5$, the model can easily generalize this concept while struggling with colors. A c_sup value larger than 0 tends to lead to more effective generalization.

The MoCo-ViT encoder also plays an important role: the original DPL model cannot generalize at all except for colors. This aligns with the poor DPL concept scores on Kand-Logic in [1].

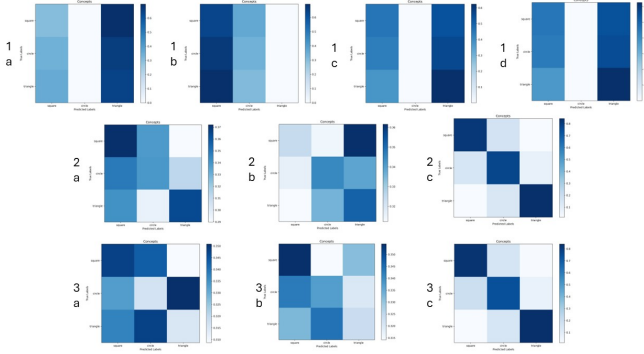


Fig. 4: Shape confusion matrices.

	Label	Concepts
True	1;0;1	"1,2,0,2,1,0;1,2,0,2,1,0;0,1,2,2,1,1" "0,1,2,2,1,1;0,2,1,2,2,2;0,2,1,2,2,2" "2,2,1,2,1,0;2,2,1,2,1,0;2,0,2,1,1,2"
1a, 1c & 1d	0;0;0	"2,2,0,0,2,2;2,2,0,0,2,2;2,2,0,0,2,2" "2,2,0,0,2,2;2,2,0,0,2,2;2,2,0,0,2,2" "2,2,0,0,2,2;2,2,0,0,2,2;2,2,0,0,2,2"
2a	0;0;0	"1,2,1,2,1,1;1,2,1,2,1,1;2,0,2,0,2,2" "2,0,2,0,2,2;0,1,0,1,0,0;0,1,0,1,0,0" "1,2,1,2,1,1;1,2,1,2,1,1;1,2,1,2,1,1"
2c	0;0;0	"0,0,1,0,0,1;0,0,1,0,0,1;0,0,1,1,0,1" "0,0,1,1,0,1;0,0,0,0,2,2;0,0,0,0,2,2" "0,0,1,0,0,1;0,0,1,0,0,1;0,0,1,1,0,1"
3a	0;0;0	"1,1,2,0,0,1;1,1,2,0,0,1;1,1,2,0,0,1" "1,1,2,0,0,1;0,0,1,1,1,2;0,0,1,1,1,2" "0,0,1,1,1,2;0,0,1,1,1,2;2,2,0,2,2,0"
3c	0;0;0	"2,1,0,2,1,0;2,1,0,2,1,0;0,1,1,1,1,2" "0,1,1,1,1,2;0,0,0,2,1,2;0,0,0,2,1,2" "0,1,1,2,1,1;0,1,1,2,1,1;0,1,1,1,1,2"

TABLE I: Qualitative results: 3 first examples from each variant's output.

C. Qualitative Results

The qualitative results I further corroborate the results seen in the confusion matrices. The MoCo-ViT encoder leads to a noticeable improvement in the quality of predictions. Still, better concept representation does not equate to better predicted y labels. This could be explained by the model not correctly generalizing the rules. These results suggest that further downstream processing is required.

IV. LIMITATIONS

The project is constrained by a number of limitations. First, some of the choices during training may not have been well-informed enough, due to the lack of comprehensive hyperparameter sweeping and the low number of runs (only 1 run for $c_{sup}=0.5$). Moreover, the current DPL training script does not make use of the validation set, and tracks training performance on the train loss instead. The validation loss would be a better metric of model training effectiveness, independent of the training set. Most importantly, the lack of

hyperparameter sweeping also caused the author to mistakenly conclude that 100 epochs is better than 10, while some results may show otherwise (see Fig. 5, 6).

Secondly, the current MoCo implementation might not be the most optimal, since the momentum encoder is not directly trained on DPL. Due to a lack of expertise, the project is limited to pretraining the base encoder to replace the 4-layer convolutional encoder. Based on the results, unsuccessful rule deduction may have also created an obstacle in improving label accuracy and F1.

V. CONCLUSION

The experiment successfully improved concept representation in DPL, although largely due to a stronger encoder. The role of MoCo pretraining is complementary – it improves performance, but neither significantly nor consistently. The concept supervision value also implies a trade-off in concept representation: higher values increase deviation and do not always reduce concept collapse. The results highlight a need for better image encoders and more effective rule-deduction systems to fully leverage the strength of neuro-symbolic models.

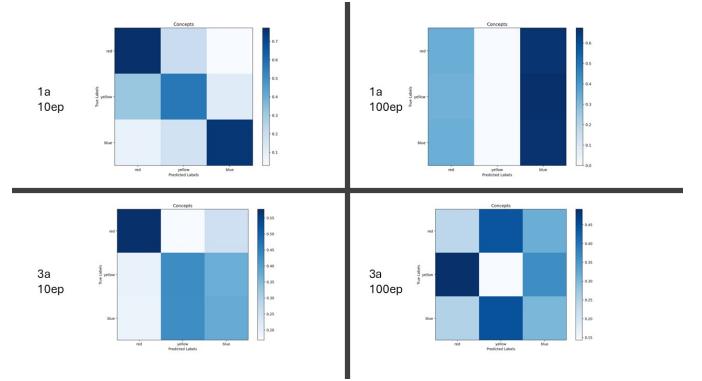


Fig. 5: Color matrices for 10 vs. 100 epochs.

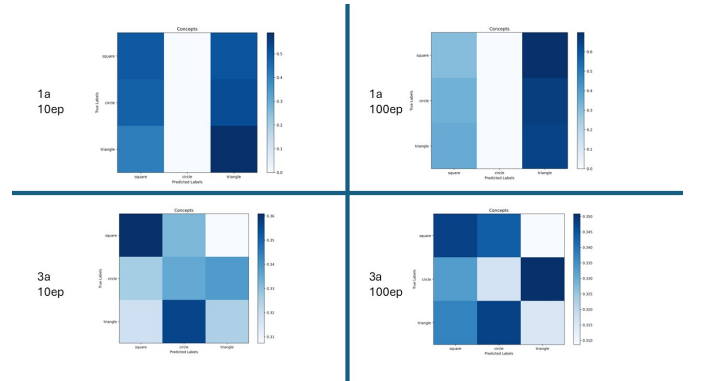


Fig. 6: Shape matrices for 10 vs. 100 epochs.

REFERENCES

- [1] S. Bortolotti, E. Marconato, T. Carraro, P. Morettin, E. van Krieken, A. Vergari, S. Teso, and A. Passerini, "A Neuro-Symbolic Benchmark Suite for Concept Quality and Reasoning Shortcuts," arXiv preprint arXiv:2406.10368, 2024.

- [2] X. Chen, S. Xie, and K. He, “An empirical study of training self-supervised vision transformers,” in Proc. IEEE/CVF Int. Conf. Comput. Vis., pp. 9640–9649, 2021.
- [3] F. Dalla Serra, G. Jacenków, F. Deligianni, J. Dalton, and A. Q. O’Neil, “Improving image representations via MoCo pre-training for multimodal CXR classification,” in Proc. Annu. Conf. Med. Image Understand. Anal. (MIUA), pp. 623–635, 2022.
- [4] R. Manhaeve, S. Dumancic, A. Kimmig, T. Demeester, and L. De Raedt, “DeepProbLog: Neural probabilistic logic programming,” Adv. Neural Inf. Process. Syst., vol. 31, 2018.
- [5] E. Marconato, S. Bortolotti, E. van Krieken, A. Vergari, A. Passerini, and S. Teso, “Bears make neuro-symbolic models aware of their reasoning shortcuts,” arXiv preprint arXiv:2402.12240, 2024.
- [6] H. Müller and A. Holzinger, “Kandinsky patterns,” Artif. Intell., vol. 300, p. 103546, 2021.
- [7] H. Sowrirajan, J. Yang, A. Y. Ng, and P. Rajpurkar, “MoCo pretraining improves representation and transferability of chest X-ray models,” in Medical Imaging with Deep Learning (MIDL), pp. 728–744, 2021.
- [8] Tensorflow, “Explore overfitting and underfitting,” TensorFlow, 2017. https://www.tensorflow.org/tutorials/keras/overfit_and_underfit

VI. APPENDICES

	nomoco		
	run 1 csup 0	run 2 csup 0	run 1 csup 0.5
acc	0.33 ± 0.02	0.33 ± 0.01	0.45 ± 0.07
accy	0.51 ± 0.01	0.51 ± 0.01	0.50 ± 0.00
f1Mc	0.25 ± 0.01	0.25 ± 0.01	0.37 ± 0.10
f1My	0.50 ± 0.02	0.50 ± 0.01	0.33 ± 0.00
CIs	0.69 ± 0.04	0.71 ± 0.05	0.40 ± 0.25
CShape	0.33 ± 0.00	0.33 ± 0.00	0.33 ± 0.21
CColor	0.33 ± 0.00	0.33 ± 0.00	0.07 ± 0.13
	mocoft		
	run 1 csup 0	run 2 csup 0	run 1 csup 0.5
acc	0.32 ± 0.02	0.33 ± 0.03	0.42 ± 0.05
accy	0.51 ± 0.01	0.52 ± 0.02	0.50 ± 0.00
f1Mc	0.31 ± 0.02	0.31 ± 0.03	0.38 ± 0.05
f1My	0.43 ± 0.04	0.46 ± 0.05	0.33 ± 0.00
CIs	0.07 ± 0.13	0.00 ± 0.00	0.09 ± 0.13
CShape	0.00 ± 0.00	0.00 ± 0.00	0.07 ± 0.13
CColor	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
	mocopre		
	run 1 csup 0	run 2 csup 0	run 1 csup 0.5
acc	0.33 ± 0.03	0.34 ± 0.03	0.49 ± 0.13
accy	0.51 ± 0.01	0.51 ± 0.00	0.50 ± 0.01
f1Mc	0.32 ± 0.02	0.33 ± 0.03	0.43 ± 0.16
f1My	0.39 ± 0.07	0.49 ± 0.02	0.36 ± 0.04
CIs	0.04 ± 0.05	0.02 ± 0.04	0.27 ± 0.33
CShape	0.00 ± 0.00	0.00 ± 0.00	0.20 ± 0.27
CColor	0.00 ± 0.00	0.00 ± 0.00	0.13 ± 0.16

TABLE II: Detailed scores of the seven metrics across the three DPL variants.